



كلية هندسة الذكاء الاصطناعي
FACULTY OF ARTIFICIAL INTELLIGENCE ENGINEERING

الجامعة السورية الخاصة
SPU SYRIAN PRIVATE UNIVERSITY

تطبيق للرعاية بالحيوانات الأليفة

اعداد:

مجد باسل مقصوصة – محمد امير الحموي

اشراف:

م. رانيا رجب

2026



كلية هندسة الذكاء الاصطناعي
FACULTY OF ARTIFICIAL INTELLIGENCE ENGINEERING

الجامعة السورية الخاصة
SPU SYRIAN PRIVATE UNIVERSITY

Pet Care App

Prepared by:

Amir Al Hamwi - Majd Bassel Maksousa

Supervised by:

Eng. Rania Rajab

2026

الملخص:

يتناول هذا المشروع تطوير نظام متكامل لرعاية الحيوانات الأليفة على شكل تطبيق موبايل، يهدف إلى تسهيل إدارة احتياجات الحيوانات الأليفة اليومية وتحسين تجربة المالكين في العناية بصحة حيواناتهم. يركز النظام على ثلاثة محاور رئيسية، وهي: متجر إلكتروني

لبيع أغذية ومنتجات العناية بالحيوانات الأليفة، نظام ملفات شخصية للحيوانات يتيح تسجيل بياناتها الصحية وتتبع مواعيد اللقاحات والأدوية والتغذية، بالإضافة إلى نظام حجز مواعيد إلكتروني يربط المستخدمين بالأطباء البيطريين ويعرض أوقاتهم المتاحة.

تواجه أنظمة رعاية الحيوانات الأليفة الحالية عدة مشكلات، من أبرزها تشتت الخدمات بين تطبيقات متعددة، وضعف تكامل البيانات الصحية، وصعوبة متابعة المواعيد الطبية بشكل منتظم، إضافة إلى غياب حلول موحدة تجمع بين التسوق والرعاية الصحية والحجز البيطري في منصة واحدة. وانطلاقاً من ذلك، يهدف هذا المشروع إلى دراسة تأثير توحيد هذه الخدمات ضمن تطبيق واحد، بهدف تحسين مستوى العناية بالحيوانات الأليفة، وتسهيل متابعة صحتها، وتطوير تجربة المستخدم من حيث السرعة والموثوقية وسهولة الاستخدام.

يتناول المشروع تصميم وتطوير تطبيق موبايل يعمل على أنظمة الهواتف الذكية، مع تحديد نطاقه في فئة المستخدمين من مالكي الحيوانات الأليفة والأطباء البيطريين. ويتبع المشروع المنهج التحليلي في دراسة الأنظمة المشابهة وتحليل متطلباتها ونقاط القوة والضعف فيها، كما يعتمد على قواعد بيانات مصممة مسبقاً لإدارة بيانات المستخدمين والحيوانات والمنتجات والمواعيد. تم تطوير الواجهة الأمامية باستخدام إطار العمل Flutter، بينما تم بناء الواجهة الخلفية باستخدام Laravel مع اعتماد نمط معمارية MVC، واستخدام أدوات حديثة في تصميم الواجهات وإدارة قواعد البيانات وواجهات البرمجة التطبيقية (API).

يقدم هذا المشروع في نتيجته نظاماً متكاملاً وفعالاً لرعاية الحيوانات الأليفة، يتميز بسهولة الاستخدام، وتنظيم البيانات الصحية، وتكامل الخدمات، مما يساهم في تحسين جودة العناية بالحيوانات الأليفة وتوفير حل رقمي عملي يلبي احتياجات المستخدمين.

Abstract:

This project focuses on developing an integrated pet care system in the form of a mobile application, aimed at facilitating the management of daily pet needs and enhancing pet owners' experience in maintaining their pets' health and well-being. The system is built around three main components: an online store for purchasing pet food and care products, a pet profile management system that allows users to record pet details and track vaccination, medication, and feeding schedules, and an appointment booking system that connects users with veterinarians and displays their available time slots.

Current pet care systems face several challenges, including fragmented services across multiple applications, poor integration of pet health records, difficulty in regularly tracking medical appointments, and the lack of a unified platform that combines shopping, health management, and veterinary booking services. Therefore, this project aims to study the impact of integrating these services into a single mobile application in order to improve pet healthcare management, enhance user convenience, and develop a more efficient and reliable digital solution.

The scope of the project includes the design and development of a mobile application targeting pet owners and veterinarians. The project follows an analytical methodology by examining existing pet care applications and identifying their strengths and weaknesses. It relies on pre-designed databases to manage user, pet, product, and appointment data. The front-end of the application is developed using Flutter, while the back-end is implemented using the Laravel framework, adopting the Model–View–Controller (MVC) architectural pattern. Modern tools and technologies are utilized for user interface design, database management, and API integration.

As a result, this project delivers an integrated and efficient pet care system characterized by ease of use, organized health data management, and seamless service integration, contributing to improved pet care quality and providing a practical digital solution that meets users' needs.

Contents

figures

tables

abbreviations

Section 1: Project Introduction.

Chapter 1: General Introduction.

1-1- Project Overview and Background:

The pet care industry is currently undergoing a significant digital transformation driven by a shift in how owners perceive their pets—not merely as animals, but as core family members requiring high-quality, organized care. Historically, the management of a pet's daily needs, ranging from acquiring essential supplies to maintaining health records and scheduling clinic visits, has been a fragmented process relying on manual tracking or multiple disconnected platforms. This project introduces a comprehensive Pet Care Application designed to serve as a unified digital ecosystem that addresses these inefficiencies. Developed using the Flutter framework for a seamless cross-platform mobile experience and supported by a robust Laravel backend, the system integrates an e-commerce pet store, personalized pet profiles with automated health and medication reminders, and a streamlined veterinary appointment booking module. By consolidating these essential services into a single interface, the project aims to simplify the responsibilities of pet ownership, ensuring that users can provide a near-full experience of everything their pets require through a professional and accessible academic-grade solution.

1-2- Problem Statement:

The current pet care landscape suffers from two primary issues:

- **Fragmented Management:** Pet owners currently lack a centralized system to manage their pets' needs, forcing them to rely on manual tracking or multiple disconnected platforms for health records, supply shopping, and service bookings.
- **Inefficiency in Care Compliance:** The absence of an automated reminder system often leads to forgotten vaccinations, missed medication schedules, and difficulties in maintaining consistent veterinary consultations.

Solving these problems is of significant value because it replaces outdated manual methods with an intelligent, all-in-one solution. By consolidating these services, the project reduces the risk of health complications for pets due to oversight and saves the owner considerable time and effort, making the journey of pet ownership more organized and professional.

1-3- Project Objectives:

1-3-1 Main Objective:

The primary objective of this project is to develop an integrated, reliable, and secure Pet Care Application that consolidates a commercial pet store, automated health profile management, and a veterinary appointment scheduling system into a single digital ecosystem.

1-3-2 Sub-Objectives:

To realize the main objective, the system aims to achieve the following measurable technical and functional targets:

- **Centralized Database Integration:** Build a unified relational database using Laravel to bridge the gap between e-commerce operations, user pet records, and clinic appointment availability.
- **Automated Notification Engine:** Implement a dependable scheduling worker to deliver time-sensitive health and vaccination reminders to users based on their custom pet profile data.
- **Optimized Network Performance:** Deliver a highly responsive mobile experience where essential application pages and network states load efficiently, aiming for average API response times under 3 seconds.
- **User-Centric Interface Design:** Structure an intuitive, accessible, and clean user interface (UI) using Flutter to ensure smooth navigation across all age groups and technical backgrounds.

1-4 Project Scope, Constraints, and Assumptions:

1-4-1 Project Scope:

The scope of this project is explicitly bound to the development of a cross-platform mobile application for end-users, managed through a central backend infrastructure.

- **In-Scope Elements:**
 - A fully functional cross-platform mobile application developed using Flutter for customers and pet owners.
 - A central backend administration system developed with Laravel to manage products, view appointment schedules, and update database configurations.
 - Essential modules including user authentication, a product catalog with shopping cart functionalities, pet profile registration with automated notification queues, and a veterinary clinic booking system

1-4-2 Project Constraints:

The execution of this project is bound by the following environmental and resource constraints:

- **Time Constraint:** The project must be entirely designed, built, tested, and documented within the academic timeline designated by the university.
- **Resource and Financial Constraints:** Development is limited to open-source tools and free hosting tiers, with zero budget for premium third-party APIs or professional production servers.
- **Data Availability:** Due to privacy regulations and limited local access, the application relies on mock and simulated data for veterinary clinics and products rather than direct live integration with actual clinical databases.

1-4-3 Assumptions:

To establish a stable baseline for development, the following factors are assumed to be true:

- **Target Audience Familiarity:** It is assumed that the end-users (pet owners) possess basic digital literacy and are comfortable navigating mobile applications on Android or iOS devices.
- **Infrastructure Availability:** It is assumed that the target users have access to a stable internet connection necessary to fetch live store listings, manage records, and schedule clinic appointments

1-5 Project Contribution:

While individual applications exist for e-commerce or clinic bookings, this project contributes to the field of applied software engineering by introducing a highly integrated, multi-service digital ecosystem tailored specifically for pet care management. The primary innovations and contributions of this system include:

- **Unified Service Architecture:** The project breaks the current market fragmentation by seamlessly combining an e-commerce platform, an automated healthcare tracker, and a real-time veterinary booking system into a single, cohesive user experience, eliminating the need for users to navigate multiple disconnected applications.
- **Proactive Care Compliance:** Through the implementation of a structured background worker system, the application transitions pet data from a passive storage model to an active notification engine, directly improving pet welfare by calculating and enforcing strict vaccination and medication timelines.

1-6 Research Methodology and Scientific Methods:

To ensure academic rigor and systematic execution, this thesis utilizes a combination of Descriptive-Analytical Research and the Applied Engineering Method. The research process is structured into three main scientific phases:

- **The Descriptive and Diagnostic Phase:** This phase relies on a descriptive approach to investigate the current challenges faced by pet owners. By gathering operational requirements and reviewing existing commercial and academic literature, a clear diagnosis of the market fragmentation problem is established.
- **The Applied Engineering Phase:** This phase implements an inductive software engineering approach to design and implement a concrete solution. It utilizes conceptual modeling tools, architectural pattern designs, and relational data analysis to systematically build the Flutter and Laravel systems based on the verified requirements.
- **The Empirical Evaluation Phase:** The final stage applies a quantitative, experimental method to test the resulting system. It relies on structured test cases, execution performance measurements, and system usability evaluations to measure the application's stability and success against the baseline objectives established in the introduction.

7-1 Document Organizational Structure:

This thesis is systematically organized into structured chapters to present a comprehensive view of the project, moving from initial concept to full implementation and evaluation:

- **Chapter 1: General Introduction:** Establishes the project background, identifies the core problem statement, outlines the system objectives, defines the scope, and outlines the research methodology.
- **Chapter 2: Project Management and Development Methodology:** Details the software engineering lifecycle chosen for development, outlines the project timeline using a Gantt chart, and assesses potential technical and operational risks.
- **Chapter 3: Theoretical Foundation and Literature Review:** Examines existing systems, analyzes modern cross-platform mobile and web backend technologies, and highlights the technical gap this system addresses.
- **Chapter 4: Requirements Engineering and Analysis:** Identifies stakeholders, formalizes functional and non-functional system requirements, and presents conceptual modeling using UML diagrams.
- **Chapter 5: System Architecture and Design:** Outlines the high-level system architecture, details the relational database entity-relationship design, and explains API security configurations.
- **Chapter 6: Implementation Environment:** Describes the software, hardware, and operating systems utilized to build, host, and run the complete system.

- **Chapter 7: Software Implementation and Code Architecture:** Illustrates the folder structure of the mobile client and backend server, accompanied by code snippets of core business logic.
- **Chapter 8: System Testing and Validation:** Outlines the testing strategy, detailing test cases and validation results for functionality, security, and interface stability.
- **Chapter 9: Results and Discussion:** Evaluates the final operational application against the primary project objectives and analyzes performance indicators.
- **Chapter 10: Conclusion and Future Work:** Summarizes the achievements of the development process, reflects on technical limitations faced, and suggests future improvements.

1-8 Basic Definitions and Concepts:

To ensure clarity throughout this thesis, the primary technological frameworks, architectural patterns, and domain-specific concepts utilized in this system are summarized below:

- **Cross-Platform Mobile Development:** A software engineering approach that allows a single codebase to compile into native applications for multiple mobile operating systems, specifically Android and iOS.
- **Flutter:** An open-source UI software development kit (SDK) created by Google that uses the Dart language to render high-performance, visually consistent applications across multiple platforms.
- **Laravel:** An open-source PHP web framework based on the Model-View-Controller (MVC) pattern, providing robust built-in tools for database management, authentication, and secure RESTful API routing.
- **RESTful API:** An architectural design style that utilizes standard HTTP requests (GET, POST, PUT, DELETE) to transfer structured JSON data statelessly between a mobile client and a web backend.
- **State Management (Provider):** An architectural pattern used to manage and synchronize data lifecycles within a user interface; in Flutter, the **Provider** package listens to state updates and selectively rebuilds specific widgets.
- **Automated Notification Queue:** A backend architectural mechanism that stores background tasks sequentially, allowing automated jobs (such as health reminders) to execute asynchronously without disrupting primary server traffic.

Chapter 2: Project Management and Development Methodology.

2-1 The Chosen Development Methodology and Justifications:

To ensure efficiency, flexibility, and steady progress, this project was developed using the Agile Development Methodology, specifically leveraging an Iterative and Incremental Lifecycle model.

Unlike the traditional Waterfall method, which requires complete phase finalization before moving forward, the Agile framework allowed the project to be divided into separate, manageable cycles (iterations). This approach is highly suited for a decoupled full-stack application because the Laravel backend API services and the Flutter mobile user interface components can be engineered, connected, and tested incrementally.

The selection of the Agile methodology was driven by several practical engineering factors:

- **Parallel Development:** It allowed for simultaneous work on the backend database relations and the corresponding frontend application states, ensuring that core modules were built together.
- **Continuous Integration and Refactoring:** Developing in loops made it easier to continuously refactor code, such as modifying data-fetching services or refining custom widgets when local emulator environments presented connection challenges.
- **Risk Reduction:** Testing smaller modules at the end of each iteration ensured that bugs—such as asynchronous network lag or broken database migrations—were identified and fixed early in the lifecycle rather than during final assembly.

2-2 Project Timeline:

To execute the project in a structured manner, the development lifecycle was mapped out across a two-month period, beginning in late February 2026 and concluding in May 2026. Adhering to the Agile framework, the timeline was broken down into distinct, requirement-focused sprints to implement core functionalities incrementally. Notably, while technical features were developed sequentially, the academic documentation process was carried out continuously in parallel to ensure accuracy.

The detailed chronological schedule, sprint breakdowns, and simultaneous workflows are visually mapped out in the Gantt chart below (Figure 2.1).

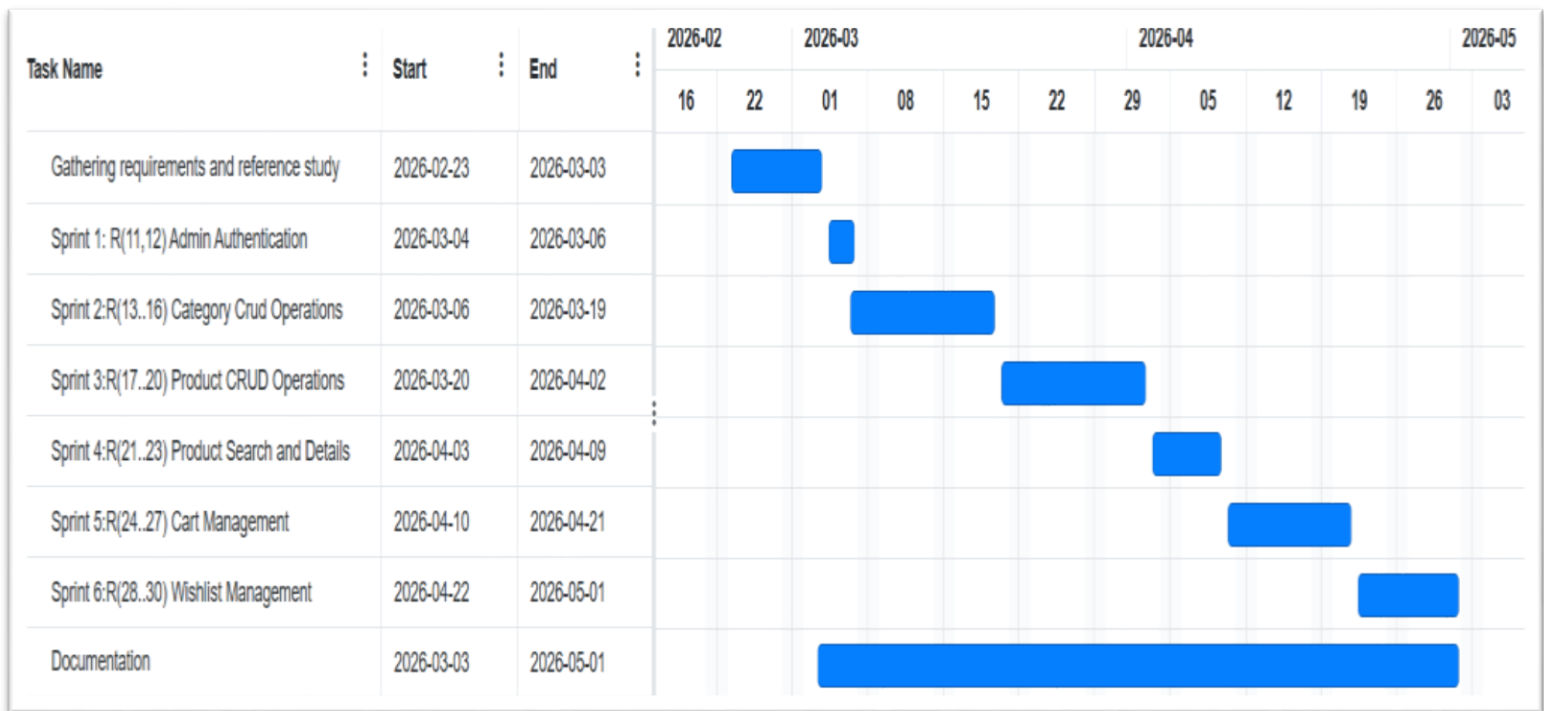


Figure 2.1

2-3 Risk Management:

Managing an agile project within a fixed academic timeline introduces several operational and technical risks. To prevent project delays and ensure system stability, potential hazards were identified early, and proactive mitigation strategies were established.

The primary risks encountered during the project lifecycle, along with their corresponding solutions, are outlined below:

- **Technical Integration Risk (API Disconnection):**
 - Risk: Potential connectivity failures or data-sync delays between the Flutter mobile client and the Laravel backend server during local emulation.
 - Fix: Comprehensive API endpoints were fully documented and tested independently using Postman before client-side integration, ensuring the data transfer logic was stable.
- **Scope Creep and Time Constraint Risk:**
 - Risk: Running out of time before completing core features due to the strict academic deadline and the complexity of managing an e-commerce system alongside appointment scheduling.
 - Fix: Adhering strictly to the prioritized sprint schedule (as seen in Figure 2.1). Non-essential secondary features were deferred to post-graduation development, protecting the core delivery timeline.
- **Data Availability Risk:**

- Risk: The inability to access real-world, live product inventories or official veterinary clinic databases.
- Fix: Developing localized seeders within Laravel to populate the system with realistic mock data, simulating authentic retail and clinical environments for testing.

2-4 Project Team Roles and Responsibilities:

2-5 Project Quality Assurance:

To ensure the delivery of a stable, reliable, and secure system within an agile framework, a structured quality assurance strategy was implemented based on three core practices:

- **Clear Acceptance Criteria:** Every functional feature within a sprint was bound to explicit operational conditions before being marked as complete (e.g., a user registration is only successful if the backend validation passes and returns a secure token).
- **Manual Testing and Payload Inspection:** API endpoints were systematically validated using Postman to verify proper JSON response structures and HTTP status codes before client-side binding.
- **Defensive Coding and Exception Handling:** Comprehensive try-catch blocks and error-boundary widgets were built into the Flutter frontend to prevent application crashes during unexpected network drops or invalid backend payloads

2-6 Project Management Tools:

The coordination, tracking, and task management of this project were organized digitally using centralized development tools to maintain strict adherence to the sprint timeline. The following tools were utilized:

- **GitHub (Repositories & Projects):** Utilized as the primary ecosystem for project management. **GitHub Projects** was deployed as a digital Kanban board to visualize the product backlog, where tasks were systematically tracked through "To Do," "In Progress," "In Review," and "Done" columns. This centralized issues, sprint tasks, and codebase updates within a single platform.
- **Postman:** Deployed as the central tool for API documentation, automated payload testing, and environment variable management to verify seamless communication between the mobile client and backend server.

2-7 Monitoring and Continuous Evaluation Mechanisms:

To ensure the project remained aligned with both technical standards and academic expectations, continuous monitoring was maintained through structured feedback loops:

- **Academic Supervisor Progress Reviews:** Scheduled bi-weekly evaluation sessions were held with the academic supervisor to demonstrate current sprint increments, review implemented architectural patterns, and receive critical feedback on thesis documentation.
- **Sprint Review and Self-Evaluation:** At the conclusion of each development cycle (as mapped in Figure 2.1), an internal audit was performed to assess completed features against the backlog, identify performance bottlenecks, and adjust the scope of subsequent sprints if necessary.

2-8 Software Configuration Management (SCM):

Version control and code configuration management were systematically enforced using **Git** hosted on a remote **GitHub repository (amirrdeveloper11/petapp)**. To manage a full-stack system efficiently, the repository utilizes a unified monorepo structure, isolating the decoupling layers into two distinct root directories:

- **/back:** Dedicated entirely to the Laravel ecosystem, containing the API routing, database migrations, and backend core models.
- **/front:** Dedicated entirely to the Flutter mobile application, containing the native asset configurations and the UI codebase initializing from `front/lib/main.dart`.

A strict branching strategy was enforced across this unified repository to ensure that frontend and backend development proceeded safely without destabilizing the system:

- **main Branch:** Serves as the primary production-ready baseline. This branch contains only stable, fully verified code that has passed both Laravel endpoint testing and Flutter state validations.
- **dev (Development) Branch:** Acts as the central integration environment. Feature branches from both the `/front` and `/back` directories were merged here first to verify that UI widgets and API payloads communicated without breaking changes.
- **feature/ Branches:** Isolated, temporary branches created for individual sprint requirements (e.g., `feature/front-cart-ui` or `feature/back-product-crud`). Once a feature met its explicit acceptance criteria, a pull request was initiated to safely merge the code back into the dev branch.

Section 2: Theoretical Study and Analysis.

Chapter 3: Theoretical Foundation and Literature Review.

3-1 Modern Technologies and Frameworks in the Filed:

This project leverages modern, production-grade frameworks to deliver a secure backend infrastructure and a high-performance mobile user interface. By utilizing an API-driven architecture, the system isolates data management from visual rendering. This section explores the technical features, execution mechanics, and architectural advantages of the selected technologies: Laravel for the backend engine and Flutter for the cross-platform mobile client.

3-1-1 The Backend Framework: Laravel (PHP):

The entire server-side application logic and data management layer (/back) are engineered using Laravel, an advanced open-source PHP framework. Built on the Model-View-Controller (MVC) design pattern, Laravel serves as a robust RESTful API engine through several key mechanisms:

- **Eloquent Object-Relational Mapping (ORM):** Laravel abstracts complex SQL database interactions into expressive, object-oriented models. Database tables (such as Categories, Products, Cart, and Wishlist items) are managed as native PHP objects, ensuring secure data handling and preventing SQL injection vulnerabilities.
- **RESTful Routing and API Resources:** Laravel includes a dedicated, lightweight routing layer optimized specifically for API development. It utilizes API Resources to act as a transformation layer, efficiently serializing database model objects into structured, web-ready JSON payloads consumed by the mobile interface.
- **Built-in Security and Request Validation:** The framework enforces centralized validation rules on all incoming HTTP requests before they touch the database. This ensures that user data, authentication tokens, and product transactions are structurally validated and secure at the server level.

3-1-2 The Frontend Mobile SDK: Flutter (Dart):

The user-facing mobile client application (/front) is compiled using Flutter, an open-source UI software development kit created by Google. Flutter is designed to compile high-performance applications from a single codebase through unique architectural features:

- **Ahead-of-Time (AOT) Native Compilation:** Flutter utilizes the Dart programming language, which compiles source code directly into native ARM and Intel machine code for both iOS and Android. This eliminates the need for an intermediate JavaScript interpreter bridge.
- **Independent Graphics Engine:** Unlike hybrid frameworks that rely on native system web views or platform UI controls, Flutter draws every pixel of its user interface independently using its own rendering engine. This guarantees that the pet care app's layout, custom store widgets, and navigation workflows remain visually identical across all mobile devices.
- **Reactive Component Tree:** Every interface element in Flutter is a widget structured in a hierarchical tree. The application updates dynamically by managing these widget lifecycles, ensuring that any user interaction—such as adding a pet item to a cart—efficiently triggers visual updates only where necessary.

3-2 Reference Study:

3-2-1 Systems Used for Comparison:

I. iPetSyria: A dedicated mobile application ecosystem operating in Syria that focuses on veterinary product sales and features a QR-enabled smart collar registry for tracking lost pets.

II. Koki.sy (Koki Store) & Aleef Store: A traditional web-based e-commerce platform in Syria designed for browsing and purchasing pet food, accessories, and maintenance supplies via a browser interface.

3-2-2 Strengths and Weaknesses:

Strengths	Weaknesses
Most platforms provide localized product catalogs tailored to the specific availability within the Syrian market.	Some platforms use a website instead of a cross-platform mobile application for easier everyday use.
Most platforms use a rating system for products.	Most platforms focus only on selling products instead of a full customer experience
Most platforms run smoothly without any noticeable bugs.	Most platforms transmit complete uncompressed database records instead of optimized data payloads.

3-3 Research Gap and Proposed Innovation:

Based on the comprehensive reference study of existing local platforms, a critical gap has been identified within the domestic pet care software market. While current systems handle isolated operational tasks, they suffer from two major unresolved limitations:

1. **Functional Fragmentation:** Existing solutions force the user to navigate fragmented environments. A user must access one platform strictly for retail e-commerce transactions (koki.sy) and completely separate local mechanisms or closed applications for pet safety or tracking. No single local system provides a centralized, cohesive customer experience.
2. **Architectural Rigidity:** Current systems utilize tightly coupled, monolithic server architectures. This results in heavy network data-over fetching, high device memory consumption, and volatile session-state management that risks losing user cart data upon unexpected client crashes or connection drops.

The Proposed Innovation:

To bridge these technical and functional gaps, this project introduces a unified, high-performance **API-Decoupled Full-Stack Mobile Application**. The core innovation of our system lies in the strategic separation of duties between frontend execution and backend business logic:

1. **Unified Customer Experience:** The application synthesizes an all-in-one digital ecosystem, integrating robust commercial retail workflows (product catalogs, persistent shopping carts) alongside advanced care data synchronization within a single user interface.
2. **Decoupled API Architecture:** By isolating a headless Laravel RESTful API engine from a natively compiled Flutter cross-platform user interface, the system eliminates traditional web bloat. The server exclusively transmits optimized, lightweight JSON text payloads, drastically reducing network data consumption and eliminating interface rendering lag on budget mobile devices.

3-4 Theoretical Framework and Reference Theories:

The architecture of the proposed system is built upon three established computer science principles:

3-4-1 Representational State Transfer (REST Architecture):

The client application and the backend server communicate via a stateless REST API model over HTTPS. Being stateless means the server does not store user session memories. Instead, every request from the app carries its own authentication token, reducing server workload and allowing the system to easily scale up to support more users.

3-4-2 API Decoupled Software Paradigm:

Unlike traditional monolithic websites where the database and visual screens are processed together on a single server, this project completely separates them. The backend functions strictly as a data provider, processing queries and sending back raw, lightweight JSON text payloads. This shifts the heavy visual rendering work entirely to the user's mobile device hardware.

3-4-3 Reactive State Management Pattern:

To ensure high performance on mobile devices, the frontend application uses a reactive state management pattern (the Provider model). When data changes—such as a user adding an item to their shopping cart—the app does not reload the entire screen. Instead, it instantly updates only the specific affected visual component, keeping the interface smooth and reducing device memory consumption.

3-5 Modern and Future Trends in the Field:

This section explores upcoming technological advancements that can be integrated into the system to enhance its scalability and user experience:

3-5-1 On-Device Machine Learning:

Traditional mobile systems process data-heavy tasks, like analyzing pet health records or processing search images, entirely on cloud servers. A major modern trend is shifting this workload to the client's device using framework tools like TensorFlow Lite. Processing smart pet data locally on the user's phone reduces server costs, provides instant feedback, and allows basic health tracking features to work completely offline.

3-5-2 Generative AI and Intelligent Customer Support:

The integration of Large Language Models (LLMs) represents a massive shift in customer interaction. Instead of navigating static text menus or waiting for manual responses from administrators, modern apps leverage specialized conversational agents. Future implementations of this platform can connect the Flutter client to automated AI assistants capable of parsing complex user descriptions about pet behaviors or symptoms to offer immediate, highly accurate care recommendations.

Chapter 4: Requirements Modeling and Analysis.

4-1 Feasibility Study:

Before initiating the development phase, a feasibility study was conducted to evaluate the practicality, resource requirements, and constraints of building the proposed full-stack mobile application. This evaluation ensures the project is viable from both a technical and financial standpoint.

4-1-1 Technical Feasibility:

The technical feasibility evaluates whether the development team possesses the necessary tools, skills, software packages, and hardware components to deliver the application successfully:

- **Software Ecosystem & Libraries:** Both selected frameworks are open-source and possess mature ecosystems. The Laravel ecosystem provides native database migration tracking, robust ORM systems, and built-in security middleware. The Flutter framework provides pre-compiled material UI widgets and reliable state-management packages (the Provider model), eliminating the need to write complex native device layouts from scratch.
- **Hardware and Development Environments:** Standard consumer-grade computers are fully capable of executing local database environments, compiling the codebase, and running mobile device emulators.
- **Technical Expertise:** The development requirements align directly with software engineering principles taught academically. Learning curves for Dart (Flutter) and PHP (Laravel) are manageable, and extensive documentation is freely available. Therefore, the project is highly feasible technically.

4-1-2 Financial Feasibility:

The financial feasibility outlines the operational expenses, infrastructure overhead, and software licensing fees required to build and host the system:

- **Development Tools and Licenses:** All core software components—including the Laravel backend framework, the Flutter frontend framework, the MySQL relational database engine, and the VS Code development environments—are completely open-source and free to use. There are zero initial software licensing costs.
- **Hosting and Server Infrastructure:** During the development and academic defense phases, the system runs entirely on a local machine (localhost), resulting in zero hosting costs. Transitioning to production in the local market would require a basic Virtual Private Server (VPS) package, which requires a minimal, predictable monthly infrastructure budget.

- **Operational Devices:** Testing utilizes existing personal mobile smartphones and local computers, eliminating any secondary hardware acquisition costs. Consequently, the financial investment required is low, making the project highly feasible.

4-2 Requirement Gathering:

To build a system that solves genuine market issues rather than superficial needs, a multi-layered requirements gathering methodology was implemented. The system's features were derived from a systematic combination of market gap analysis and structured user feedback.

4-2-1 Domain Observation and Market Analysis:

The foundational core requirements of this application were established through a collaborative technical evaluation of the current local market by the development team. By conducting an architectural audit of existing local pet service platforms, several consistent bottlenecks were identified:

- **Functional Fragmentation:** Users are forced to use entirely separate channels or manual methods to handle retail shopping and pet care tracking.
- **Interface Rendering Lag:** Heavy page loading and UI micro-stutters occur over local mobile data networks due to outdated monolithic web setups.

To transition these development observations into verified requirements, a validation survey was created.

4-2-2 Validation Survey:

An electronic questionnaire titled "Pet Care and Shopping Mobile Application Survey" was designed to objectively test and validate these initial market observations with actual end-users. The questionnaire targets pet owners in Syria, focusing on three specific diagnostic areas:

1. **User Experience Friction:** Measuring user frustration levels regarding functional fragmentation across current unlinked services.
2. **Infrastructure Capability:** Capturing user experiences with latency, UI stuttering, and high data consumption over local 3G/4G connections.
3. **Feature Demand Prioritization:** Gauging consumer interest in a unified mobile application that provides smooth, fast-loading interfaces for both product browsing and care tracking.

By deploying this survey, the development observations are directly cross-referenced against live user experiences, ensuring that the system's finalized functional specifications are rooted firmly in authentic consumer requirements.

4-3 Stakeholder Identification and Needs Analysis:

To design a highly targeted software architecture, it is essential to define the distinct groups of users interacting with the platform. This section identifies the primary stakeholders and analyzes their specific functional expectations:

4-3-1 End Users (Pet Owners):

The primary consumers of the mobile application require a fast, intuitive, and reliable interface to manage their daily pet requirements. Their core needs include:

- **Unified Accessibility:** A single application environment where they can seamlessly browse retail pet products and track animal health or care schedules without needing multiple disconnected platforms.
- **Responsive Performance:** A fluid user interface that loads rapidly over standard mobile networks without lagging, draining battery life, or stuttering on budget devices.

4-3-2 Service Providers (Pet shops and Veterinarians):

This group consists of local business operators and veterinary clinics who use the platform as a commercial and service channel. Their core needs include:

- **Effortless Inventory & Service Listing:** A simple, non-technical interface to quickly upload new pet products, adjust prices, or list available clinic/care appointment slots.
- **Order & Booking Visibility:** Clear, immediate notifications and tracking for incoming product orders or care appointments made by pet owners so they can fulfill them promptly

4-3-3 System Administrators:

The backend administrators are the technical managers responsible for maintaining the underlying application infrastructure, verifying security, and monitoring system health. Their core needs include:

- **Centralized Dashboard Control:** A secure administrative panel to monitor overall user activity, audit database changes, approve new service provider accounts, and manage platform-wide server configurations.
- **Data Integrity & Security Boundaries:** Robust backend validation layers to block corrupted payloads, prevent unauthorized data tampering, and protect the central database from malicious or malfunctioning client requests.

4-4 Functional Requirements:

Functional requirements describe the specific actions, operations, and tasks that the application must execute. To ensure a clear technical scope, the system's low-level functional requirements are explicitly detailed and broken down by their associated system actors:

4-4-1 User Requirements:

- **FR-01 User Login:** The user must be able to log in to the application.
- **FR-02 User Logout:** The user must be able to log out from their active session.
- **FR-03 Create New Account:** The user must be able to register a new account within the system.
- **FR-04 Edit Profile:** The user must be able to modify their personal account details.
- **FR-05 Delete Profile:** The user must be able to permanently delete their personal account.
- **FR-06 View Profile Account Details:** The user must be able to view their personal account specifications.
- **FR-07 Add Pet Profile:** The user must be able to create a new profile for their pet.
- **FR-08 Edit Pet Profile:** The user must be able to modify an existing pet profile.
- **FR-09 Delete Pet Profile:** The user must be able to permanently remove a pet profile.
- **FR-10 View Pet Profile Details:** The user must be able to view the complete details of a pet's profile.
- **FR-21 Select Specific Category:** The user must be able to select and view a single targeted product category.
- **FR-22 Search for a Product:** The user must be able to query the platform to find a particular product item.
- **FR-23 View Product Details:** The user must be able to pull up the individual details page for any product.
- **FR-24 Add Product to Shopping Cart:** The user must be able to place an inventory item into their virtual shopping cart.
- **FR-25 View Shopping Cart:** The user must be able to open and inspect their current shopping cart contents.
- **FR-26 Modify Product Quantity in Cart:** The user must be able to adjust the unit counts of an item residing inside the cart.
- **FR-27 Delete Product from Shopping Cart:** The user must be able to eject a specific product from their cart.
- **FR-28 Add Product to Wishlist:** The user must be able to save an item to their personal favorites list.
- **FR-29 View Wishlist:** The user must be able to access and review their saved favorites list.
- **FR-30 Delete Product from Wishlist:** The user must be able to clear an item off their favorites list.

4-4-2 System Administrator Requirements:

- **FR-11 Dashboard Login:** The admin must be able to securely log in to the administrative control panel.
- **FR-12 Dashboard Logout:** The admin must be able to safely log out from the control panel.
- **FR-13 Create Product Category:** The admin must be able to create new product classifications.
- **FR-14 Edit Product Category:** The admin must be able to modify existing product classifications.
- **FR-15 Delete Product Category:** The admin must be able to remove product classifications from the platform.
- **FR-17 Insert New Product:** The admin must be able to introduce a new retail item into the system inventory.
- **FR-18 Edit Product Details:** The admin must be able to adjust individual product information, pricing, or attributes.
- **FR-19 Delete Product:** The admin must be able to completely delete a specific product listing.

4-4-3 Shared Admin and User Requirements:

- **FR-16 View Product Category:** Both administrators and standard users must be able to view the available categories.
- **FR-20 View Product List:** Both administrators and standard users must be able to display the full inventory catalog of products.

4-5 Non-Functional Requirements:

While functional requirements define what the system must physically do, non-functional requirements (NFRs) specify the operational criteria, constraints, and quality attributes of the platform. These requirements ensure that the system is performant, secure, dependable, and maintainable under production workloads.

4-5-1 Performance and Scalability:

Response Latency: The REST API backend must process and return data for standard read operations (such as fetching the product list or viewing pet profiles) within less than 2 seconds under normal network conditions.

Optimized Payload Delivery: To accommodate local mobile network constraints, all media assets and JSON data structures transmitted from the server must be compressed to minimize bandwidth consumption and prevent interface stuttering.

Concurrent Session Handling: The database and server architecture must support multiple concurrent user sessions without degrading the transaction processing speeds of the shopping cart operations.

4-5-2 Security and Privacy:

Data Transmission Protection: All client-server communications, sensitive data transfers, and API endpoints must be encrypted using Transport Layer Security (TLS/HTTPS) protocols to prevent data interception.

State Verification & Access Control: The application must utilize secure, stateless JSON Web Tokens (JWT) for user sessions, ensuring that users can only access, edit, or delete data resources that explicitly belong to their authenticated accounts.

Input Sanitization: The backend ecosystem must automatically intercept and clean all inbound strings at the controller layer to isolate and block cross-site scripting (XSS) patterns or database manipulation threats.

4-5-3 Availability and Reliability:

Platform Uptime: The underlying application hosting infrastructure must maintain an operational availability target of 99.5%, minimizing unscheduled downtime during system updates or maintenance.

Fault Isolation: The system must use decoupled error-handling mechanisms so that a failure in a minor subsystem (such as a loading delay in the Wishlist component) does not crash the core application or disrupt basic product browsing.

4-5-4 Usability and Compatibility:

UI Responsiveness: The mobile user interface must dynamically adapt its rendering layout across various smartphone screen resolutions and aspect ratios without clipping text or overlapping interactive buttons.

Cross-Platform Consistency: The application must deliver a visually uniform experience on standard modern versions of both mobile operating systems.

4-6 Requirement Prioritization (MoSCoW Framework):

To ensure an efficient development cycle and establish a clear roadmap for the Minimum Viable Product (MVP), the 30 identified functional requirements are categorized using the industry-standard MoSCoW prioritization framework. This methodology divides requirements into four distinct levels of urgency: Must have, Should have, Could have, and Won't have.

4-6-1 Must Have (M):

These requirements form the mandatory, non-negotiable core engine of the platform. Without these features, the application cannot be deployed or operate. This group includes foundational user authentication, catalog setup, and product management:

- **User & Session Security:** Core login, logout, and registration actions (FR-01, FR-02, FR-03), alongside administrative dashboard entry safeguards (FR-11, FR-12).
- **Inventory Foundation:** The backend ability to create, edit, view, and remove product categories (FR-13, FR-14, FR-15, FR-16).
- **Product Catalog Control:** The fundamental operations allowing administrators to insert, modify, view, and delete physical items within the database inventory (FR-17, FR-18, FR-19, FR-20).

4-6-2 Should Have (S):

These requirements represent highly important operational capabilities that significantly improve the user experience and fulfill basic business goals. While the system can technically run without them, their absence makes the application impractical for daily consumer use:

- **Catalog Interaction:** The ability for users to select specific categories, search the inventory, and view deep product specification pages (FR-21, FR-22, FR-23).
- **Transaction Lifecycle:** Core e-commerce features allowing users to add items to a shopping cart, view cart contents, modify unit quantities, or remove items from the order list (FR-24, FR-25, FR-26, FR-27).

4-6-3 Could Have (C):

These features are classified as valuable additions that enhance user engagement and personal convenience, but they are not critical to the application's immediate core functionality. They can be safely postponed or omitted if project deadlines shift:

- **Profile Personalization:** User profile modifications, account deletions, and dedicated pet portfolio management (FR-04, FR-05, FR-06, FR-07, FR-08, FR-09, FR-10).
- **User Wishlist System:** The secondary ability to add items to a personal favorites list, display the saved favorites list, or remove items from it (FR-28, FR-29, FR-30).

4-6-4: Won't Have (W):

These requirements represent advanced features that are explicitly excluded from the current development scope of this project. Acknowledging these boundaries prevents scope creep and defines potential areas for future updates:

- **Automated Payment Gateways:** Integration with electronic payment networks or local mobile wallet APIs.

- **Real-Time Delivery Tracking:** Live GPS tracking maps for order dispatch and delivery routes.
- **Predictive Product Recommendations:** AI-driven suggestion engines based on historical user shopping patterns.